

李乐岩

3073751449@qq.com | +86 18805506898 | GitHub: <https://github.com/leyancode>

爱丁堡大学 University of Edinburgh 英国 | 2025.09 - 2026.12

高性能计算与数据科学 硕士

课程与毕业论文 2026.08 完成 | 可入职时间 2026.09.01 | 学位证书 2026.12 获得

利物浦大学 University of Liverpool 英国 | 2023.09 - 2025.06

计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

项目经历:

面向 HPC Benchmark 的 Agent 式性能分析系统 (GitHub 开源、可复现)

项目描述: 基于真实 ARCHER2 超算上的 PETSc benchmark 运行数据, 构建“确定性工具 + 可评测路由”的

Agent 式性能分析系统: 将 benchmark 结果结构化为 SQLite 数据库, 把性能分析任务封装为确定性分析工具, 用

中文规则 router 作为可验证 baseline, 通过 CLI 以中文完成最优配置查询、模式对比与扩展性报告生成; 进一步接

入 LLM function calling router, 用自建 30 问五类中文评测集对两种 router 打分——规则 83.3%、LLM 100%,

并由消融实验定位分数主要来源为 system prompt 而非模型本身; 再补齐工程化外围: FastAPI 暴露 POST

/ask、trace_id 贯穿的 JSON Lines 事件流、以及对本地笔记的词法检索 (IDF 加权, 零新依赖)。代码与评测集开

源: github.com/leyancode/hpc-benchmark-agent

技术栈: Python、SQLite、pytest、FastAPI、OpenAI SDK (function calling)、PETSc、MPI、OpenMP、Slurm、Git、Linux

工作内容:

- 设计分层架构: 数据导入层 (CSV → SQLite)、指标层 (speedup / parallel efficiency)、工具层 (最优配置查询、模式对比、扩展性报告)、检索层 (本地笔记词法检索)、路由层 (规则与 LLM function calling 双实现、输出严格同构)、可观测层 (trace_id + 事件流), 以及 CLI 与 FastAPI 两个平级且互不依赖的进程边界; 各层输入输出合同由 229 项自动化测试钉死, 全量测试离线可跑。
- 实现中文规则 router: 关键词意图分类 + 问题规模解析 (阿拉伯数字、“100 万”混写、中文数字“一百万”), 缺参数主动追问、超范围明确拒答; 输出与 LLM tool call 严格同构 (intent + 参数), 两种 router 因此可共用一套评测集, CLI 上切换只需改一行 dispatch。
- 封装确定性分析工具: 单规模最优配置查询、MPI-only 与 hybrid 模式对比 (数据只含 ranks/threads, 故采用保守标签 mpi_like / hybrid_like)、扩展性报告生成; 同一数据库生成的 Markdown 报告字节级一致 (刻意不含时间戳), 使“文档是否漂移”退化为一次 git diff 检查。
- 基于真实 ARCHER2 数据验证: 100 万未知数规模下单核 245.7s → 128 核 3.866s (speedup 63.6、并行效率 49.7%), 识别出 2/4 核超线性加速 (cache 效应, 效率 147% / 126%) 与 32 核 hybrid 配置的效率坍塌 (0.33), 并由规则自动生成中文解读段落。
- 接入 LLM function calling router: 设计 tool schema (description 写明工具边界; unknowns 刻意可选——模型省略参数即「追问」、不调用工具即「拒答」); 实现防御式响应解析, 把未调用工具、幻觉工具名、arguments 非合法 JSON 三类异常统一收敛为显式 unknown, 绝不猜参数以避免「静默算错」; API 失败向上抛出而非回退规则 router, 以免污染评测。单元测试全部经 FakeLLM 依赖注入离线运行, 无需 API key。
- 建立评测机制与对比结论: 自建 30 问五类中文评测集 (正常 / 追问 / 拒答 / 数字陷阱 / 规则已知会错的对抗题), 期望

值为 tool call 而非答案文本，harness 接受任意 router 函数、对其实现无知；规则 router 83.3%、LLM router 100%，而消融实验显示同一模型不加 system prompt 仅 63.3%——准确率主要由工具说明书与策略 prompt 决定；报告如实标注局限（题量 30、单模型单次运行）。

- 实现结构化 tracing：每次提问生成唯一 trace_id，路由决策与工具调用以 JSON Lines 事件流记录（ask_start → route_result → tool_call / tool_result → answer），可完整回放一次 agent 执行；脱敏收敛在单一出口点（绝对路径降级为文件名、疑似密钥字段一律 redacted），并由扫描测试强制保证。
 - 以 FastAPI 薄封装对外暴露 POST /ask：只做协议转换、不重复实现任何业务逻辑，请求体约束写进类型（非法 router 取值由 Pydantic 拦为 422）；缺 API key 映射为 503（服务端依赖未就绪）而非 500，与 CLI 把同一异常映射为退出码 2 各自独立——同一个库、两个进程边界各自翻译错误语义。新增该边界时既有文件一行未改，可作为分层是否真正解耦的实测。
 - 实现最小 RAG（词法检索）：对本地 HPC 笔记做 IDF 加权词重叠检索（中英混合分词、中文取相邻 bigram、top-k 与阈值过滤），纯标准库、零新依赖——刻意不用 embedding 与向量库，换来离线、确定性与可逐条解释的命中理由；检索不到即返回空、不注入上下文，绝不退而求其次返回“最像的一条”；未开启检索时模型看到的 messages 与评测时逐字节一致并由测试锁死，使既有评测结论保持可比。
 - 工程纪律：pytest 全量离线可跑、不依赖 API key（key 走环境变量，永不入库）；README demo 命令逐条可复现（已用全新 clone 从零验证）；以“公开声称 == 已实现功能”作为文档与简历的验收约束。
-

多智能体 Actor-Critic 方法中的价值函数分解研究（本科毕业设计、GitHub 开源）

项目描述：在 MAPPO 的 critic 上分别接入 VDN 的加性分解与 QMIX 的单调 mixing，做单变量对照：三个变体共用同一套 runner / replay buffer / shared RNN actor 与全部 PPO 超参，差异只在 critic 分支。结论与直觉相反——表达能力更强的 critic 没有赢：稀疏奖励、部分可观测的 Predator-Prey（300k steps）上，最简单的加性分解是唯一学到正回报的方法（终值 ≈ 150 ），单体 critic 始终为负（ $-300 \rightarrow -30$ ），单调 mixing 的 critic loss 起手即达 5.4×10^4 、200k 步后二次发散。代码与实验图表开源：github.com/leyancode/marl-value-factorisation

技术栈：Python、PyTorch、PyMARL、MAPPO、VDN、QMIX、CTDE、PPO、PettingZoo / MPE2、Sacred、pytest

工作内容：

- 实现三种 critic 变体并对齐为单变量对照：单体 $V(s)$ 、加性分解 $\sum_i V_i(o_i)$ 、以 global state 为条件、由 hypernetwork 生成的单调 mixing network；三者共用同一条更新流程（target critic 构造 $TD(\lambda)$ 目标 → critic 做 masked MSE 更新 → 以 λ -return 减 V 作为 advantage 喂给 PPO clipped surrogate），使 critic 结构成为唯一自变量；采用 CTDE 范式，执行期 actor 只依赖局部观测，全局状态仅用于训练 critic。
- 定位 MAPPO-QMIX 不收敛的根因：其 TD 目标沿用 value-based QMIX 的 $\max_a Q_i$ （贪婪动作），而 actor 是 on-policy，两者相减得到的 td_error 并非合法 advantage 而是系统性偏乐观的量，可解释实验中反复出现的“回报冲过理论最优值后崩塌”；另定位 hypernet 经 softplus 生成 mixing 权重（ $\text{softplus}(0) \approx 0.69$ ，初始权重并不接近零）对局部 Q 的梯度做乘性放大，是 critic loss 起手即达 5×10^4 的直接原因。

- 在三个 payoff 结构不同的 matrix game 与 Predator-Prey 上完成对比评估：单调性约束 $\partial Q_{tot}/\partial Q_i \geq 0$ 在“最优解要求两个 agent 选择不同动作”的 mixed-motive payoff 上与任务结构冲突，QMIX 变体确实最差；并以解析计算佐证 baseline 停住的位置对应共享 actor 未能打破 agent 间对称性，而非 critic 的分解方式（标注为待消融验证的假设，未作为结论）。
 - 主动记录负结果与自身局限：去掉单调约束的 non-monotonic mixer 持续梯度爆炸、未能在预算内收敛，如实写入论文而非略去；整理阶段进一步指出三个 learner 实际都只产生一个共享的标量 advantage，故分解带来的收益来自表示与梯度路径，而非真正的 per-agent credit assignment，并据此提出 COMA 式 counterfactual baseline 作为下一步单变量实验。
 - 2026 年整理为可复现的作品集仓库：修正原 Predator-Prey wrapper 中合作边界不清的问题（改为只将 3 个 predators 暴露给 learner，prey 由可设 seed 的固定随机策略驱动）；用 uv.lock 锁定 CPU 版 PyTorch 使实验无需 GPU 即可复现，统一训练入口，补充 pytest 并接入 GitHub Actions CI；基于 Oxford WhiRL 的 PyMARL (Apache-2.0) 二次开发，在 UPSTREAM.md 中明确区分上游代码与个人实现范围。
-

技术能力

编程语言与工程开发：熟悉 Python、C/C++，掌握 SQL 基础，了解 JavaScript、HTML、CSS；可用 Python 完成 AI 应用原型、数据处理、日志解析、实验自动化与模型训练流程。

AI Agent 与 LLM 应用：了解 LLM Agent 基本架构，包括 tool calling、任务分解、RAG 检索增强生成、prompt engineering、记忆管理、多轮对话状态管理和 Markdown 报告生成；具备将大模型能力与外部工具、数据库和自动化脚本结合、并对其做离线评测的项目开发经验。

RAG 与检索：在本项目中实现了词法检索（IDF 加权词重叠、中英混合分词、top-k 与阈值过滤），并刻意未引入向量检索——结构化 benchmark 数据的问答由 SQL 完成，准确性与可验证性优于向量召回，检索只用于补充解释性上下文；了解向量路线的完整流程（文档解析与切分、embedding 生成、FAISS / Chroma 索引、top-k 语义检索与上下文拼接），以及它与词法检索在可解释性、冷启动成本和同义词召回上的取舍。

后端与系统设计基础：在本项目中实做了 FastAPI 接口层，包括请求体校验、异常到 HTTP 状态码的映射（依赖未就绪 503、参数非法 422）与 trace_id 贯穿的结构化事件日志；了解 REST API、HTTP、客户端与服务端交互、任务调度、限流、幂等性和可观测性等后端工程概念。

数据库与缓存：了解 MySQL / SQLite 使用及索引、事务、锁、查询优化等概念（本项目中实际设计了 benchmark 表结构与查询索引）；了解 Redis 常用缓存场景与 TTL、缓存穿透 / 击穿、一致性问题。

机器学习与数据分析：熟悉 PyTorch，具备深度学习和多智能体强化学习算法实现与实验评估经验；熟悉 Pandas、NumPy、Matplotlib，可完成结构化数据处理、指标计算、实验日志分析和可视化。

工具与开发环境：熟悉 Linux、Bash、Git、VS Code、Docker 基础操作；具备用自动化脚本管理实验、处理日志并构建可复现开发流程的经验。

论文发表

Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis.
International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.